Day-24 Python 90 Days Challenges

Encapsulation means **binding data and methods together inside a class** and controlling how data is accessed or modified.

Basic Idea

Data + Methods = Encapsulation

1. Encapsulation keeps related data and behavior inside one class.
2. It helps protect data from direct unwanted changes.
3. Python does not have strict private variables like Java or C++.
4. Python uses naming conventions to show access level.
5. Encapsulation is commonly handled using:
    1. Public attributes
    2. Protected attributes
    3. Private/name-mangled attributes
    4. Getter and setter methods
    5. @property

PYTHON CODE

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())
```

Output: 1500

Here, __balance is not directly accessed from outside. It is accessed using methods.

Public, Protected, and Private Members

Python uses naming conventions for access control.

Access Level Table

Nishchal

Type	Syntax	Meaning	Example
Public	name	Can be accessed normally	self.name
Protected	_name	Meant for internal use	self._salary
Private / Name mangling	__name	Avoid accidental outside access	self.__pin

Example 1

```
PYTHON CODE
class Student:
    def __init__(self):
        self.name = "Aman"
        self._marks = 85
        self.__grade = "A"

student = Student()
print(student.name)
print(student._marks)
```

Output: Aman
85
Direct access to __grade will fail:
print(student.__grade)
Output: AttributeError: 'Student' object has no attribute '__grade'

Important Point

Double underscore does not make data fully private.
It performs name mangling to avoid accidental access.
Internally, Python changes:
__grade -> _Student__grade
So technically it can still be accessed, but it should not be used directly.

Getter and Setter Methods

Getter and setter methods are used to read and update private data safely.

CodeWithNishchal

```python
def get_value(self):
    return self.__value

def set_value(self, value):
    self.__value = value
```

1. Getter method reads private data.
2. Setter method updates private data.
3. Setter can validate data before updating.
4. This protects the object from invalid values.

Example 1

PYTHON CODE

```python
class Student:
    def __init__(self, marks):
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def set_marks(self, marks):
        if marks >= 0:
            self.__marks = marks
        else:
            print("Marks cannot be negative")
student = Student(80)
student.set_marks(90)
print(student.get_marks())
```

Output: 90

Encapsulation Using @property

Nishchal

@property allows a method to behave like an attribute.

Syntax

```python
@property
def attribute_name(self):
    return value
```

1. @property is a clean way to control access to data.
2. It allows validation before changing data.
3. It supports getter and setter behavior.
4. It makes code look simple while still protecting data.

Example 1

PYTHON CODE

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if value < 0:
            raise ValueError("Marks cannot be
negative")
        self._marks = value

student = Student(85)
student.marks = 95
print(student.marks)
```

Output: 95

14.2.2 Inheritance

Nishchal

Inheritance allows one class to reuse the properties and methods of another class.

Syntax

```python
class ChildClass(ParentClass):
    statement
```

Explanation

1. Inheritance supports code reuse.
2. The parent class is also called base class or superclass.
3. The child class is also called derived class or subclass.
4. The child class can use parent class attributes and methods.
5. The child class can also define its own attributes and methods.
6. The child class can override parent methods.
7. Inheritance represents an **is-a** relationship.

Basic Example

PYTHON CODE

```python
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")
dog = Dog()
dog.eat()
dog.bark()
```

Output: Eating
Barking

Inheritance Flow

Parent class

hNishchal

|
v
Child class
|
v
Child class can reuse parent features

Why Inheritance is Used

Use	Meaning
Code reuse	Child class reuses parent code
Extensibility	Child class can add new features
Maintainability	Common logic stays in parent class
Polymorphism	Same method can behave differently
Real-world modeling	Represents is-a relationship

Example:
Dog is an Animal
Car is a Vehicle
Student is a Person

Types of Inheritance in Python

Python supports these major types of inheritance:

Type	Meaning
Single inheritance	One child inherits from one parent
Multilevel inheritance	Child inherits from parent, and another child inherits from that child
Hierarchical inheritance	Multiple child classes inherit from one parent
Multiple inheritance	One child inherits from multiple parents
Hybrid inheritance	Combination of two or more inheritance types

CodeWithNishchal

Single Inheritance

Single inheritance means one child class inherits from one parent class.

Syntax

```python
class Parent:
    pass

class Child(Parent):
    pass
```

Example 1

```
PYTHON CODE
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

dog = Dog()
dog.eat()
dog.bark()
```

Output: Eating
Barking

CodeWithNishchal

Multilevel Inheritance

Multilevel inheritance means a class inherits from a child class, forming a chain.

Syntax

```python
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

Example 1

PYTHON CODE

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

class Puppy(Dog):
    def weep(self):
        print("Weeping")

puppy = Puppy()
puppy.eat()
puppy.bark()
puppy.weep()
```

Output:
Eating
Barking
Weeping

Hierarchical Inheritance

Hierarchical inheritance means multiple child classes inherit from one parent class.

Nishchal

Syntax

```python
class Parent:
    pass

class Child1(Parent):
    pass

class Child2(Parent):
    pass
```

Example 1

```
PYTHON CODE
class Animal:
    def eat(self):
        print("Eating")

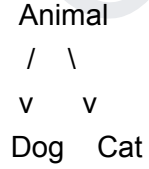
class Dog(Animal):
    def bark(self):
        print("Barking")

class Cat(Animal):
    def meow(self):
        print("Meowing")

dog = Dog()
cat = Cat()
dog.eat()
dog.bark()
cat.eat()
cat.meow()
```

Output: Eating
Barking
Eating
Meowing

Diagram



Multiple Inheritance

Multiple inheritance means one child class inherits from more than one parent class.

Code Nishchal

Syntax

```
PYTHON CODE
class Parent1:
    pass

class Parent2:
    pass

class Child(Parent1, Parent2):
    pass
```

1. Python supports multiple inheritance.
2. A child class can use methods from multiple parent classes.
3. If parents have methods with the same name, Python uses MRO.
4. MRO means Method Resolution Order.

Example 1

```
PYTHON CODE
class Father:
    def father_skill(self):
        print("Gardening")

class Mother:
    def mother_skill(self):
        print("Painting")

class Child(Father, Mother):
    def child_skill(self):
        print("Coding")

child = Child()

child.father_skill()
child.mother_skill()
child.child_skill()
```

Output:

Gardening
Painting
Coding

Diagram

Father Mother
  \ /

Nishchal

v v

Child

Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

1. Hybrid inheritance mixes different inheritance types.
2. It can include multiple, multilevel, or hierarchical inheritance together.
3. It is powerful but can become complex.
4. MRO is important in hybrid inheritance.

Example 1

PYTHON CODE

```
class Person:
    def show_person(self):
        print("Person")

class Student(Person):
    def show_student(self):
        print("Student")

class Employee(Person):
    def show_employee(self):
        print("Employee")

class TeachingAssistant(Student, Employee):
    def show_ta(self):
        print("Teaching Assistant")

ta = TeachingAssistant()
ta.show_person()
ta.show_student()
ta.show_employee()
ta.show_ta()
```

Output:

Person

Student

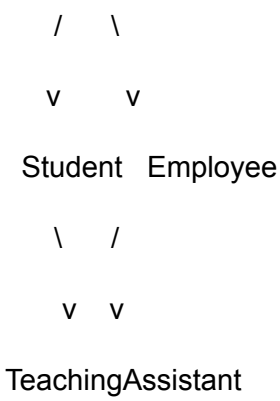
Employee

Teaching Assistant

Diagram

Person

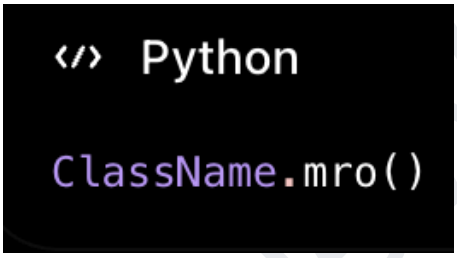
Nishchal

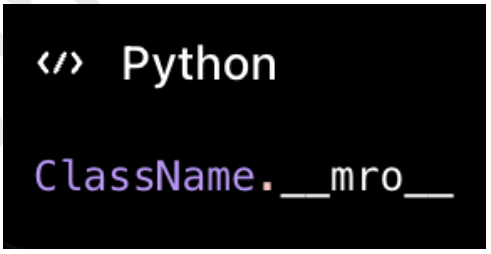


14.17 Method Resolution Order - MRO

MRO is the order Python follows while searching for methods in inheritance.

Syntax



OR


1. MRO decides which method is called first.
2. It is important in multiple inheritance.
3. Python uses the C3 linearization algorithm for MRO.
4. The search starts from the child class.
5. Then Python checks parent classes according to MRO.
6. Finally, it checks the base object class.

Example 1

PYTHON CODE
```
class A:
```

CodeWithNishchal

```
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass

obj = D()
obj.show()
print(D.mro())
```

Output: B

[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]

MRO Flow

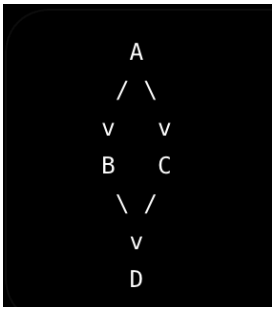
D -> B -> C -> A -> object

Since B comes before C, B.show() runs.

Diamond Problem

CodeWithNishchal

The diamond problem happens when a child class inherits from two classes that both inherit from the same parent.



1. Class B and class C both inherit from A.
2. Class D inherits from both B and C.
3. If the same method exists in multiple classes, Python uses MRO to decide.
4. Python handles this safely using MRO.
5. super() also follows MRO.

Example 1

PYTHON CODE

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass
d = D()
d.show()
```

Output: B
Because the MRO is:
D -> B -> C -> A -> object

Nishchal

Constructor in Inheritance

When a child object is created, Python runs the child class __init__ method.
If the child class does not have __init__, Python uses the parent class __init__.

Case 1: Child Has No __init__

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    pass

student = Student("Aman")
print(student.name)
```

Output: Aman

The child class uses the parent class constructor.

Case 2: Child Has Its Own __init__

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, course):
        self.course = course

student = Student("Python")
print(student.course)
```

Output:

Python

Here, parent __init__ does not run automatically because the child has its own __init__.

Nishchal

super()

super() is used to call the next method in the MRO, usually from the parent class.

Syntax

super().method_name()

Explanation

1. super() is commonly used to call parent class methods.
2. It is commonly used inside __init__.
3. It avoids directly writing the parent class name.
4. In multiple inheritance, super() follows MRO.
5. It helps avoid repeating parent class logic.

Example 1

PYTHON CODE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

student = Student("Aman", "Python")

print(student.name)
print(student.course)
```

Output:

Aman

Python

Nishchal

14.2.3 Method Overriding

Method overriding means a child class defines a method with the same name as a parent class method.

Syntax

```python
class Parent:
    def method(self):
        statement

class Child(Parent):
    def method(self):
        new_statement
```

Explanation

1. The method name is the same in parent and child.
2. The child class provides its own version.
3. When called on child object, child method runs.
4. This supports polymorphism.
5. Parent method can still be called using super().

Example 1

PYTHON CODE

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

dog = Dog()
dog.sound()
```

Output:Bark

WithNishchal

Overriding with super()

```
PYTHON CODE
class Animal:
    def sound(self):
        print("Animal sound"
class Dog(Animal):
    def sound(self):
        super().sound()
        print("Bark")
dog = Dog()
dog.sound()
```

Output: Animal sound
Bark

14.2.4 Polymorphism

Polymorphism means the same method or operation behaves differently for different objects.

Explanation

1. Polymorphism means “many forms”.
2. The same method name can work differently in different classes.
3. Python supports polymorphism through:
    1. **Method overriding**
    2. **Duck typing**
    3. **Operator overloading**

```
PYTHON CODE
class Dog:
    def sound(self):
        print("Bark")


class Cat:
    def sound(self):
        print("Meow")
animals = [Dog(), Cat()]
for animal in animals:
    animal.sound()
```

Output: Bark

Meow

WithNishchal

Polymorphism Types in Python

Type	Meaning	Example
Method overriding	Child changes parent method	Dog.sound()
Duck typing	Object is accepted if it has required method	animal.sound()
Operator overloading	Operator works differently by class	obj1 + obj2

14.2.5 Duck Typing

Duck typing means Python focuses on what an object can do, not its exact type.

Basic Idea

If an object has the needed method, Python can use it.

Example 1

PYTHON CODE

```
class Duck:
    def speak(self):
        print("Quack")

class Person:
    def speak(self):
        print("Hello")

def call_speak(obj):
    obj.speak()

call_speak(Duck())
call_speak(Person())
```

Output:
Quack
Hello

Python does not care whether the object is Duck or Person. It only checks whether speak() exists.

Nishchal

14.2.6 Abstraction

Abstraction means hiding internal implementation and showing only essential features.

Explanation

1. Abstraction focuses on what an object does.
2. It hides how the object does it internally.
3. Python supports abstraction using abstract base classes.
4. Abstract base classes are created using the abc module.
5. A class with abstract methods cannot be instantiated directly.
6. Child classes must implement abstract methods.

Example 1

PYTHON CODE

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class UpiPayment(Payment):
    def pay(self, amount):
        print("Paid", amount, "using UPI")

payment = UpiPayment()

payment.pay(500)
```

Output: Paid 500 using UPI

Important Point

Payment cannot be used directly because it has an abstract method.

This gives an error:

payment = Payment()

Output:

TypeError: Can't instantiate abstract class Payment with abstract method pay

Nishchal

Encapsulation vs Abstraction

Point	Encapsulation	Abstraction
Meaning	Protecting and controlling data	Hiding implementation details
Focus	Data access	Essential behavior
Achieved by	Private variables, methods, @property	Abstract classes, interfaces
Example	Hide __balance	Define pay() without showing payment logic

Encapsulation = How data is protected
Abstraction = How unnecessary details are hidden

14.2.7 Composition

Composition means one class contains an object of another class.

Basic Idea

Inheritance = is-a relationship
Composition = has-a relationship

1. Composition is used when one object is made of another object.
2. It represents a **has-a** relationship.
3. It is often preferred over inheritance when there is no true is-a relationship.

Example 1

```
PYTHON CODE
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
car = Car()
car.start()
```

Output:Engine started
Explanation:
Car has an Engine.
So this is composition.

WithNishchal

Association, Aggregation, & Composition

Relationship	Meaning	Example
Association	One class uses another class	Teacher teaches Student
Aggregation	One class has another, but both can exist independently	Department has Teachers
Composition	One class owns another strongly	House has Rooms

Important Difference

Point	Aggregation	Composition
Relationship	Weak has-a	Strong has-a
Child object can exist alone?	Yes	Usually no
Example	Team has Players	Car has Engine

isinstance() and issubclass()

These functions are useful when working with inheritance.

Syntax

isinstance(object, ClassName)
issubclass(ChildClass, ParentClass)

1. isinstance() checks whether an object belongs to a class.
2. It also returns True if the object belongs to a child class.
3. issubclass() checks whether one class inherits from another class.
4. Both return True or False.

Example 1

PYTHON CODE
```
class Animal:
    pass

class Dog(Animal):
    pass

dog = Dog()
print(isinstance(dog, Dog))
print(isinstance(dog, Animal))
print(issubclass(Dog, Animal))
```

Output: True
True
True

devWithNishchal

Inheritance vs Composition

Point	Inheritance	Composition
Relationship	Is-a	Has-a
Code style	Child class extends parent	Class contains another object
Example	Dog is an Animal	Car has an Engine
Reuse method	Inherit methods	Use contained object methods
Flexibility	Can become tightly connected	Usually more flexible

Use Inheritance When

Child really is a type of parent.

Example:Dog is an Animal

Student is a Person

Use Composition When

One object has or uses another object.

Example:

Car has an Engine

Computer has a Processor

Complete Inheritance Types Summary

Type	Structure	Example
Single	A -> B	Animal -> Dog
Multilevel	A -> B -> C	Animal -> Dog -> Puppy
Hierarchical	A -> B, A -> C	Animal -> Dog, Animal -> Cat
Multiple	A + B -> C	Father + Mother -> Child
Hybrid	Combination	Person -> Student/Employee -> TeachingAssistant

OOP Principles Summary

CodeWithNishchal

Principle	Meaning	Python Feature
Encapsulation	Control access to data	_name, __name, getter/setter, @property
Inheritance	Reuse parent class code	class Child(Parent)
Polymorphism	Same method, different behavior	Overriding, duck typing
Abstraction	Hide implementation details	ABC, @abstractmethod
Composition	Build one object using another	Object inside object

Important OOP Scenario Table

Scenario	Use
Need to protect data	Encapsulation
Need to reuse common code	Inheritance
Need same method with different behavior	Polymorphism
Need to force child class method structure	Abstraction
Need one object inside another	Composition
Need multiple parent classes	Multiple inheritance
Need to resolve method search order	MRO
Need parent constructor logic	super()
Need to check object/class relationship	isinstance(), issubclass()

CodeWithNishchal

14.3 Special Methods / Magic Methods / Dunder Methods